



# Orquestación de Agentes Autónomos con n8n

## Procesamiento del Lenguaje Natural

Prof. Lucas Miguel Iturriago Salas

Universidad Nacional de Colombia  
Departamento de Ingeniería Eléctrica, Electrónica y Computación

Primer Semestre - 2026



1. Sistemas de Agentes Orientados a Eventos
2. Inicialización del Agente y Paradigma ReAct
3. Grounding y Herramientas Dinámicas
4. Despliegue en Producción y MLOps



- En las sesiones previas, operamos bajo un paradigma de inferencia estrictamente secuencial y lineal.
- El flujo de control dependía de un bucle síncrono donde el estado del sistema  $S_t$  se actualizaba iterativamente a partir del token predicho  $y_t$ :

$$S_t = f(S_{t-1}, y_t)$$

- Al escalar hacia **Agentes Autónomos**, pasamos de una tubería estática a un **Grafo Dirigido de Ejecución**.
- El Agente se convierte en un sistema reactivo conducido por eventos. Cada nodo en la topología muta y transporta un payload estructurado en formato JSON.

## Definición de Arquitectura Basada en Eventos (EDA)

El LLM actúa como un motor de enrutamiento dinámico. Evalúa la semántica de la entrada y decide de manera estocástica el camino de ejecución a través de los conectores de infraestructura (I/O).

- **Abstracción de LangChain/LlamaIndex:** Sustituimos las canalizaciones rígidas de código en Python por un lienzo visual serverless.
- **Anatomía de un Evento:** En n8n, toda arista transmite un array de objetos independientes:

```
[ { "json": { "meta": { "timestamp": "..."} } } ]
```

Los LLMs están limitados por los pesos fijos parametrizados durante su entrenamiento (conocimiento estático). Para interactuar con entornos cambiantes, se requiere el bucle formal **ReAct**:

- 1 **Thought (Pensamiento)**: El modelo genera un razonamiento latente  $t_t \in \mathcal{T}$  para determinar el estado de la tarea.
- 2 **Action (Acción)**: El LLM selecciona e invoca una herramienta externa  $a_t \in \mathcal{A}$  bajo una firma sintáctica estricta (esquema JSON).
- 3 **Observation (Observación)**: El entorno ejecuta la acción y devuelve un resultado  $o_t \in \mathcal{O}$ .

Cada observación se concatena secuencialmente de vuelta en la ventana de contexto:

$$\text{Prompt}_{t+1} = [\text{Prompt}_t \parallel t_t \parallel a_t \parallel o_t]$$

La interfaz de n8n separa estructuralmente las responsabilidades cognitivas del sistema:

- **Capa de Cómputo (Model Sub-node):** Administra de forma aislada la inferencia estocástica y los hiperparámetros del backend de Google AI Studio.
- **Capa de Lógica de Negocio (Agent Node):** Configura el comportamiento maestro del sistema (*System Instruction*), gobernando las restricciones operativas.

## Beneficio de Ingeniería

Permite el intercambio en caliente del LLM subyacente (ej. de Gemini a un modelo local en Ollama) sin alterar ni una sola línea de código de las herramientas o de la personalidad del agente.



El modelo genera una distribución de probabilidad sobre el vocabulario mediante una función **Softmax** ajustada por la temperatura  $T$ :

$$P(y_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- Cuando la temperatura tiende a cero ( $T \rightarrow 0$ ), la distribución se colapsa en un vector *one-hot* sobre el logit  $z_i$  más alto.
- Se elimina la entropía estocástica (la aleatoriedad en el muestreo).
- **Justificación en Agentes:** Para invocar herramientas y mapear variables en código estructurado, no buscamos creatividad, sino **determinismo y reproducibilidad matemática**.



- Un Webhook o una API REST no poseen estado por naturaleza (*stateless*). Cada interrupción de red corre en el vacío.
- Para dotar al agente de coherencia en hilos conversacionales, conectamos el subnodo **Window Buffer Memory**.
- Implementa una estructura de datos de memoria volátil tipo **FIFO (First-In, First-Out)** de tamaño de ventana  $k$ .
- Almacena únicamente las últimas  $k$  interacciones completas, podando de forma dinámica los tensores de tokens más antiguos.
- **Optimización de MLOps:** Reduce drásticamente la latencia de procesamiento, mitiga el re-procesamiento redundante de tokens y evita la saturación de los límites del contexto del API.

## ¿Qué es Grounding (Anclaje a la Realidad)?

Es el proceso de vincular la generación de texto del modelo a fuentes de información externas verificables y en tiempo real, erradicando las alucinaciones por desactualización de pesos.

En este laboratorio implementamos dos tipos fundamentales de herramientas dinámicas:

- **Lectura de Base de Datos Local:** Nodo Google Sheets configurado bajo el recurso *Sheet within document* y la operación *Get Rows*.
- **Adquisición de Telemetría Externa:** Nodo HTTP Request configurado para enviar peticiones asíncronas GET a un endpoint REST dinámico (Open-Meteo).

## Advertencia Crítica de Ingeniería

El LLM no conoce el código de programación interno de n8n ni la URL de destino de las APIs de manera mágica. La invocación depende de la descripción semántica de la herramienta.

- El planificador del Agente lee el string inyectado en el campo **Description** del subnodo.
- Realiza un emparejamiento de intenciones en lenguaje natural con la solicitud del usuario.
- Si la semántica coincide, el orquestador extrae los argumentos e invoca la operación de red subyacente de forma transparente.

Para que el agente pase de ser un prototipo en lienzo a un microservicio integrado en la arquitectura corporativa, cambiamos los disparadores por un nodo **Webhook**:

## Configuración del Endpoint:

- **Method:** POST (Mantiene las instrucciones dentro del body encapsulado).
- **Response Mode:** *When Last Node Finishes* (Mantiene la conexión HTTP síncrona hasta concluir ReAct).

## Entornos de Red:

- **Test URL:** Escucha en caché únicamente durante la depuración activa.
- **Production URL:** Activa un proceso demonio 24/7 en la nube cuando el flujo pasa a estado **Active**.

- Al transicionar de un chat gráfico integrado a una petición HTTP cruda, la memoria del sistema pierde el identificador del remitente de forma automática.
- Para resolver el **Error 500 / No Session ID found**, debemos asumir el control del mapeo y configurarlo en la opción **Define below**.
- Realizamos un enlazado de datos (*Data Binding*) dinámico apuntando explícitamente a las claves raíz del JSON entrante del Webhook:

## Mapeo de Variables en el Lienzo

- **ID de Sesión (Memoria):** `{{ $json.body.sessionId }}`
- **Prompt del Usuario (AI Agent):** `{{ $json.body.chatInput }}`

Mapeo de la petición HTTP final para interactuar con el pipeline desde la terminal del sistema (**cURL**) o mediante código estructurado en **Python**:

## Petición mediante cURL

```
curl -X POST WEBHOOK_URL \  
-H "Content-Type: application/json" \  
-d '{"sessionId": "lucas-01", \  
    "chatInput": "¿Hay osciloscopios?"}'
```

## Petición mediante Python (Librería Requests)

```
import requests  
payload = {  
    "sessionId": "lucas-01",  
    "chatInput": "¿Hay osciloscopios?",  
}  
response = requests.post(WEBHOOK_URL, json=payload)  
print(response.json())
```



- Con el microservicio operativo, la fase final del ciclo de vida involucra la supervisión y la observabilidad a través de la pestaña de **Executions** de n8n.
- **Trazabilidad en Tiempo Real:** Permite inspeccionar de forma granular el payload JSON exacto inyectado en cada arista del grafo.
- **Auditoría de Inferencia:** Monitorea los costos de tokens por minuto (TPM), el tiempo total de respuesta (*latency scaling*) y valida el comportamiento determinista de las llamadas a las funciones.

¡El pipeline de agentes autónomos está listo para producción!